

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

Midterm Exam 1 Review

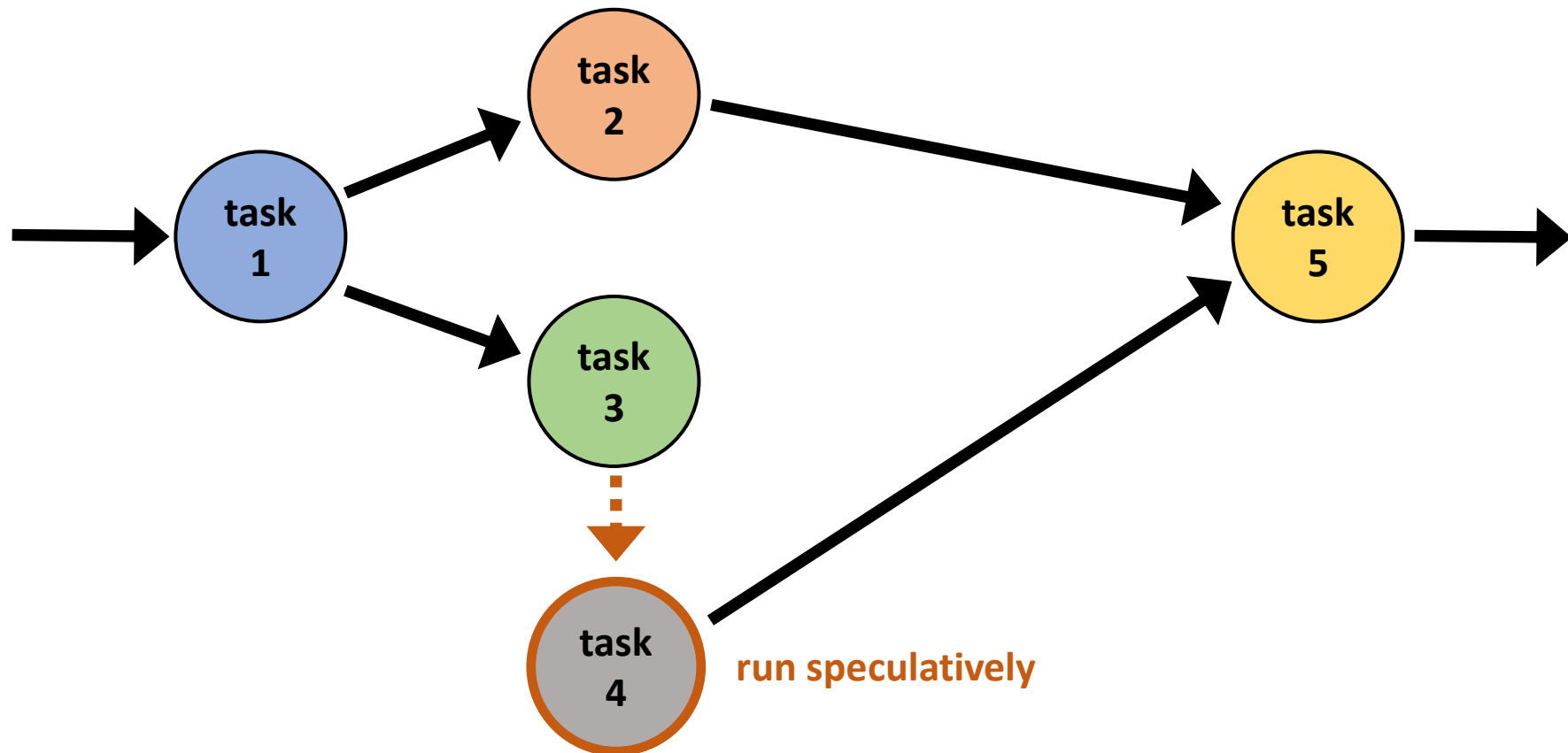
Instructor: Prof. Joshua San Miguel

Multiprocessors and Multiprogramming

Give an example of parallel execution where computation is not concurrent.

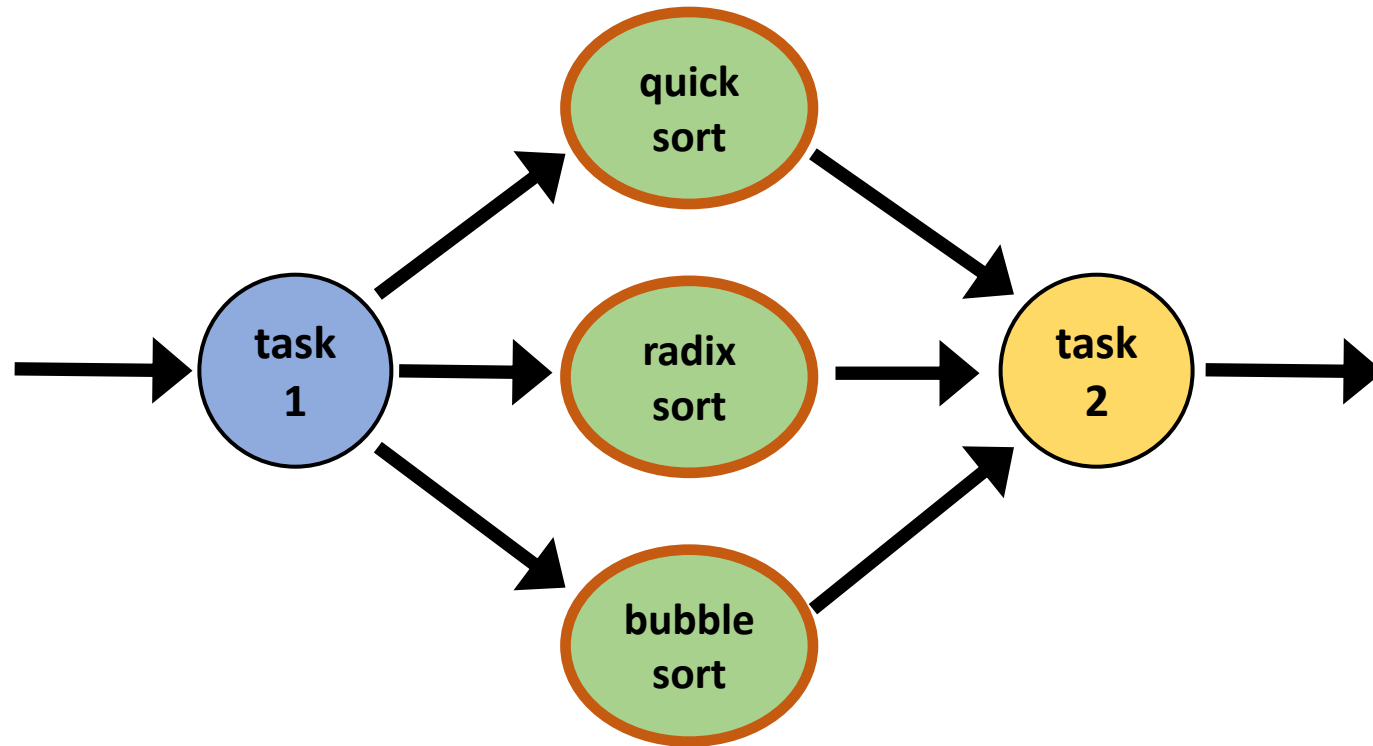
Multiprocessors and Multiprogramming

Give an example of parallel execution where computation is not concurrent.



Multiprocessors and Multiprogramming

Give an example of parallel execution where computation is not concurrent.



Multiprocessors and Multiprogramming

Consider a multi-threaded program that is 90% parallel. At least how many cores would you need to achieve an upper bound speedup within 10% of the ideal speedup (i.e., infinitely many cores)?

Multiprocessors and Multiprogramming

Consider a multi-threaded program that is 90% parallel. At least how many cores would you need to achieve an upper bound speedup within 10% of the ideal speedup (i.e., infinitely many cores)?

Upper bound speedup, ∞ cores = $1 / (1/10 + 0) = 10$

Multiprocessors and Multiprogramming

Consider a multi-threaded program that is 90% parallel. At least how many cores would you need to achieve an upper bound speedup within 10% of the ideal speedup (i.e., infinitely many cores)?

Upper bound speedup, ∞ cores = $1 / (1/10 + 0) = 10$

Upper bound speedup, n cores $\geq 90\% \times$ Upper bound speedup, ∞ cores
 $1 / (1/10 + 9/10n) \geq 9$

Multiprocessors and Multiprogramming

Consider a multi-threaded program that is 90% parallel. At least how many cores would you need to achieve an upper bound speedup within 10% of the ideal speedup (i.e., infinitely many cores)?

Upper bound speedup, ∞ cores = $1 / (1/10 + 0) = 10$

Upper bound speedup, n cores $\geq 90\% \times$ Upper bound speedup, ∞ cores
 $1 / (1/10 + 9/10n) \geq 9$

Therefore $n \geq 81$ cores

Multiprocessors and Multiprogramming

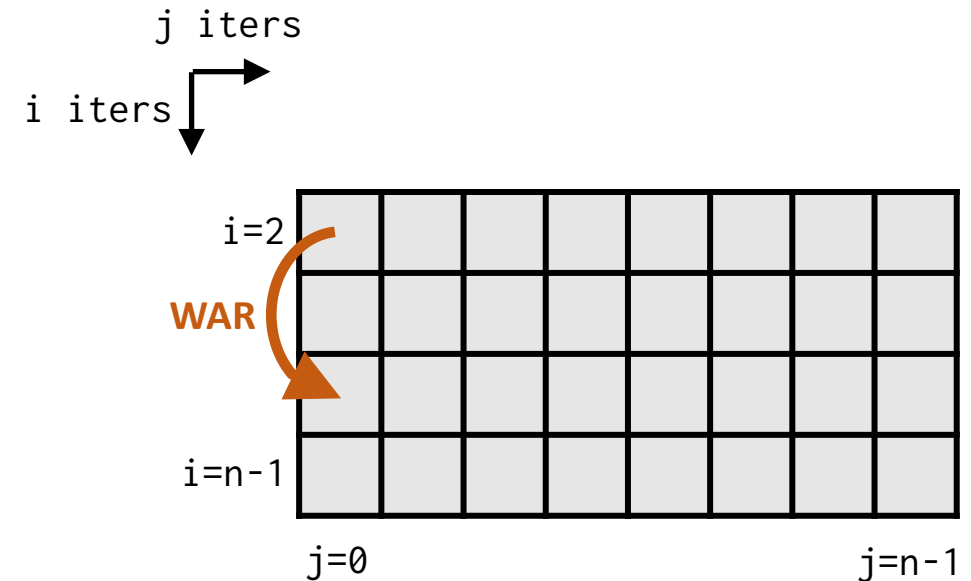
List any loop-carried dependences and state which loops are parallelizable.

```
for(int i = 2; i < n; i++) {  
    for(int j = 0; j < n; j++) {  
        a[i-2][j] = a[i][j];  
    }  
}
```

Multiprocessors and Multiprogramming

List any loop-carried dependences and state which loops are parallelizable.

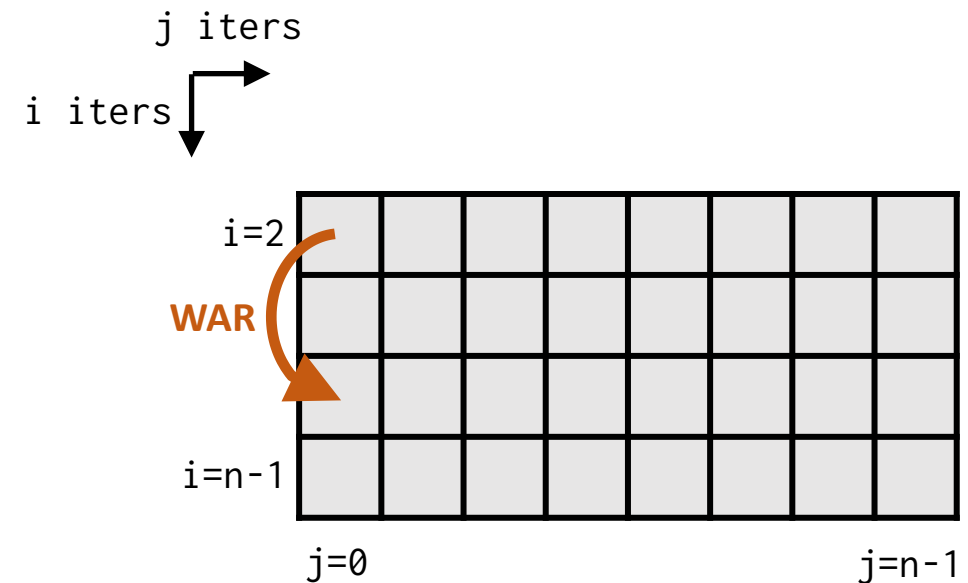
```
for(int i = 2; i < n; i++) {  
    for(int j = 0; j < n; j++) {  
        a[i-2][j] = a[i][j];  
    }  
}
```



Multiprocessors and Multiprogramming

List any loop-carried dependences and state which loops are parallelizable.

```
#pragma omp parallel for  
for(int i = 2; i < n; i++) {  
    #pragma omp parallel for  
    for(int j = 0; j < n; j++) {  
        a[i-2][j] = a[i][j];  
    }  
}
```



Synchronization

List all possible outcomes of A and B after executing all threads.

```
// init  
*x = 0; *y = 0;
```

```
// P1  
lock();  
*y = *x + 1;  
unlock();
```

```
// P2  
lock();  
*x = *y + 1;  
unlock();
```

```
// P3  
lock();  
A = *x;  
B = *y;  
unlock();
```

Synchronization

List all possible outcomes of A and B after executing all threads.

(A=0 B=0) (A=0 B=1) (A=1 B=0) (A=2 B=1) (A=1 B=2)

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
lock();
```

```
*y = *x + 1;
```

```
unlock();
```

```
// P2
```

```
lock();
```

```
*x = *y + 1;
```

```
unlock();
```

```
// P3
```

```
lock();
```

```
A = *x;
```

```
B = *y;
```

```
unlock();
```



Synchronization

List all possible outcomes of A and B after executing all threads.

```
// init
*x = 0; *y = 0;
```

```
// P1
lock();
*y = *x + 1;
unlock();
```

```
// P2
lock();
*x = *y + 1;
unlock();
```

```
// P3
A = *x;
B = *y;
```

Synchronization

List all possible outcomes of A and B after executing all threads.

(A=0 B=0) (A=0 B=1) (A=1 B=0) (A=2 B=1) (A=1 B=2) **(A=0 B=2)**

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
lock();
```

```
*y = *x + 1;
```

```
unlock();
```

```
// P2
```

```
lock();
```

```
*x = *y + 1;
```

```
unlock();
```

```
// P3
```

```
A = *x;
```

```
B = *y;
```



Synchronization

Is it possible to implement an array-based queuing lock if the only atomic instruction available is compare-and-swap (CAS)? If so, how? If not, explain.

```
r2 = CAS(addr, r1, r2):  
    { temp=r2; r2=*addr; if r2==r1 then *addr=temp; }
```

LOCK:

```
myticket = fetch-and-add(lockvar->nextticket, 1);  
while(lockvar->isready[myticket] != 1);
```

UNLOCK:

```
lockvar->isready[myticket + 1] = 1;
```


Synchronization

Is it possible to implement an array-based queuing lock if the only atomic instruction available is compare-and-swap (CAS)? If so, how? If not, explain.

```
r2 = CAS(addr, r1, r2):  
    { temp=r2; r2=*addr; if r2==r1 then *addr=temp; }
```

LOCK:

```
do {  
    myticket = lockvar->nextticket;  
} while(myticket != CAS(lockvar->nextticket, myticket, myticket+1));  
while(lockvar->isready[myticket] != 1);
```

UNLOCK:

```
lockvar->isready[myticket + 1] = 1;
```

Synchronization

Which of the following barrier implementations do not need atomic instructions?

- i. Tournament barrier
- ii. MCS tree barrier
- iii. Centralized barrier
- iv. Dissemination barrier
- v. Software combining tree barrier

Synchronization

Which of the following barrier implementations do not need atomic instructions?

- i. **Tournament barrier**
- ii. MCS tree barrier
- iii. Centralized barrier
- iv. Dissemination barrier
- v. Software combining tree barrier

Synchronization

Which of the following barrier implementations do not need atomic instructions?

- i. **Tournament barrier**
- ii. **MCS tree barrier**
- iii. Centralized barrier
- iv. Dissemination barrier
- v. Software combining tree barrier

Synchronization

Which of the following barrier implementations do not need atomic instructions?

- i. **Tournament barrier**
- ii. **MCS tree barrier**
- iii. ~~Centralized barrier~~
- iv. Dissemination barrier
- v. Software combining tree barrier

Synchronization

Which of the following barrier implementations do not need atomic instructions?

- i. **Tournament barrier**
- ii. **MCS tree barrier**
- iii. ~~Centralized barrier~~
- iv. **Dissemination barrier**
- v. Software combining tree barrier

Synchronization

Which of the following barrier implementations do not need atomic instructions?

- i. **Tournament barrier**
- ii. **MCS tree barrier**
- ~~iii. Centralized barrier~~
- iv. **Dissemination barrier**
- ~~v. Software combining tree barrier~~

Cache and Memory Hierarchy

Assume a program exhibits perfect temporal locality (e.g., program with memory blocks A, B and C accesses them in such a sequence: A, ..., A, B, ..., B, C, ..., C). Which of the following types of cache misses can be observed when running this program?

- i. Conflict misses
- ii. Capacity misses
- iii. Cold (compulsory) misses

Cache and Memory Hierarchy

Assume a program exhibits perfect temporal locality (e.g., program with memory blocks A, B and C accesses them in such a sequence: A, ..., A, B, ..., B, C, ..., C). Which of the following types of cache misses can be observed when running this program?

- i. ~~Conflict misses~~
- ii. Capacity misses
- iii. Cold (compulsory) misses

Cache and Memory Hierarchy

Assume a program exhibits perfect temporal locality (e.g., program with memory blocks A, B and C accesses them in such a sequence: A, ..., A, B, ..., B, C, ..., C). Which of the following types of cache misses can be observed when running this program?

- i. ~~Conflict misses~~
- ii. ~~Capacity misses~~
- iii. Cold (compulsory) misses

Cache and Memory Hierarchy

Assume a program exhibits perfect temporal locality (e.g., program with memory blocks A, B and C accesses them in such a sequence: A, ..., A, B, ..., B, C, ..., C). Which of the following types of cache misses can be observed when running this program?

- i. ~~Conflict misses~~
- ii. ~~Capacity misses~~
- iii. **Cold (compulsory) misses**

Cache and Memory Hierarchy

Describe one software technique and one hardware technique for reducing the impact of false sharing in caches.

Cache and Memory Hierarchy

Describe one software technique and one hardware technique for reducing the impact of false sharing in caches.

Software:

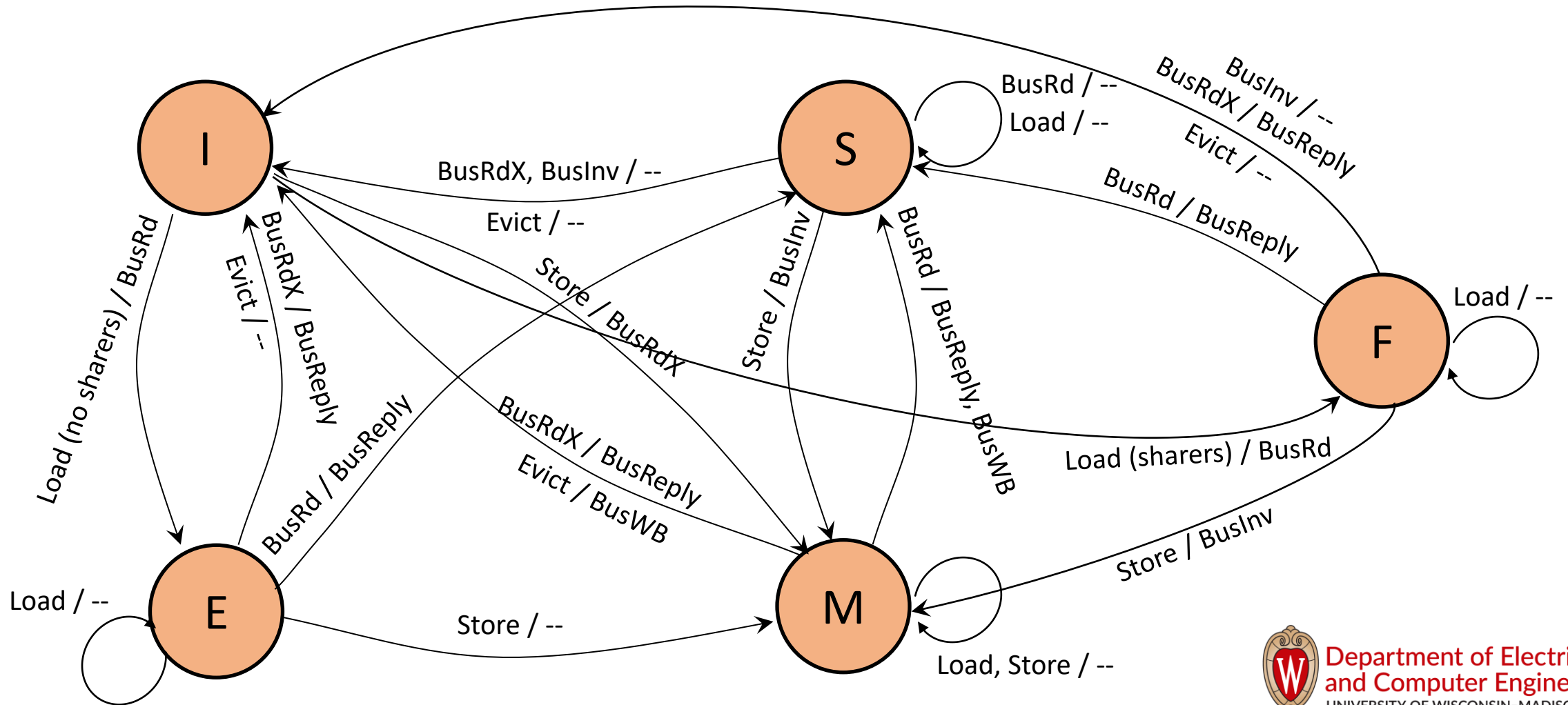
- Padding
- Block scheduling (OpenMP)

Hardware:

- Reducing cache block size
- Coherence decoupling

Snooping-Based Coherence

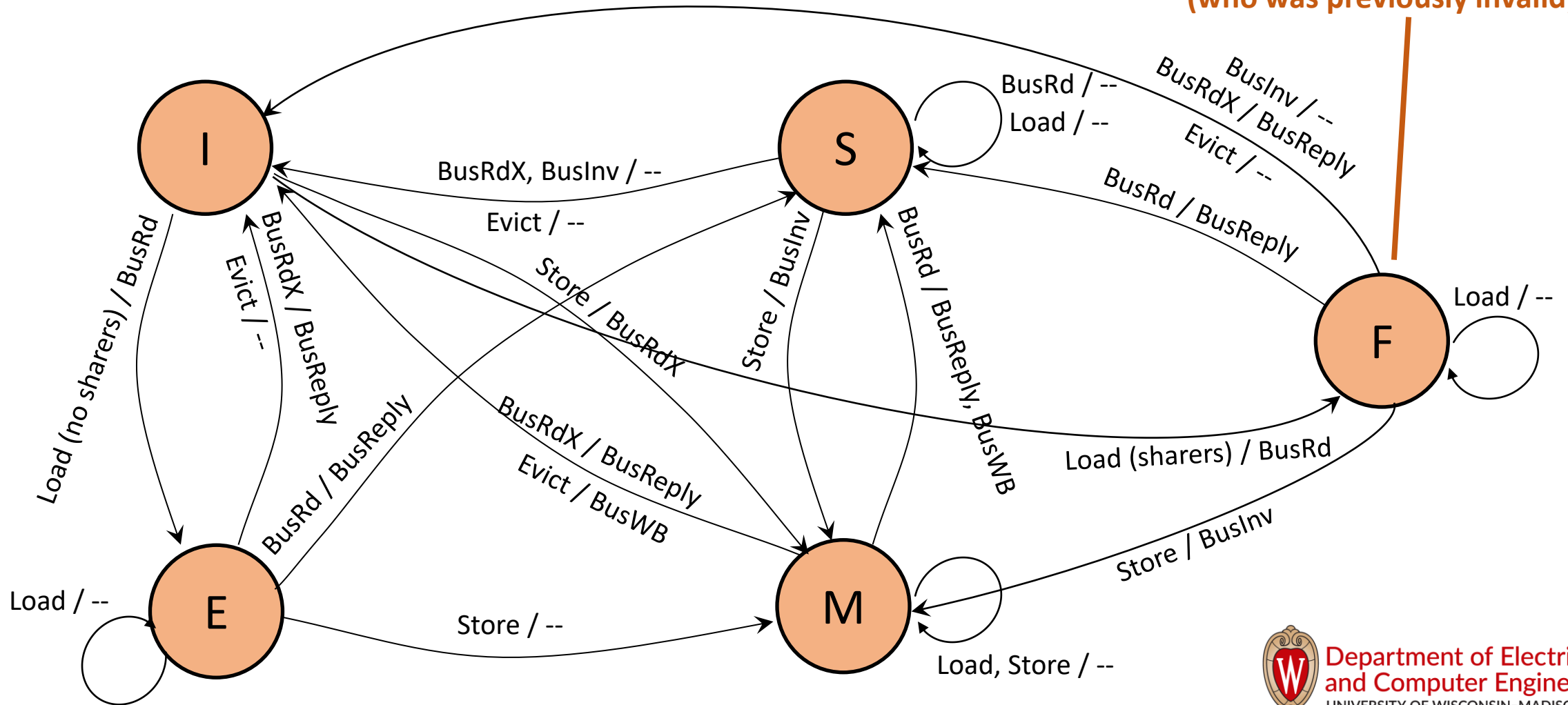
In MESIF below, why is there no transition from S to F?



Snooping-Based Coherence

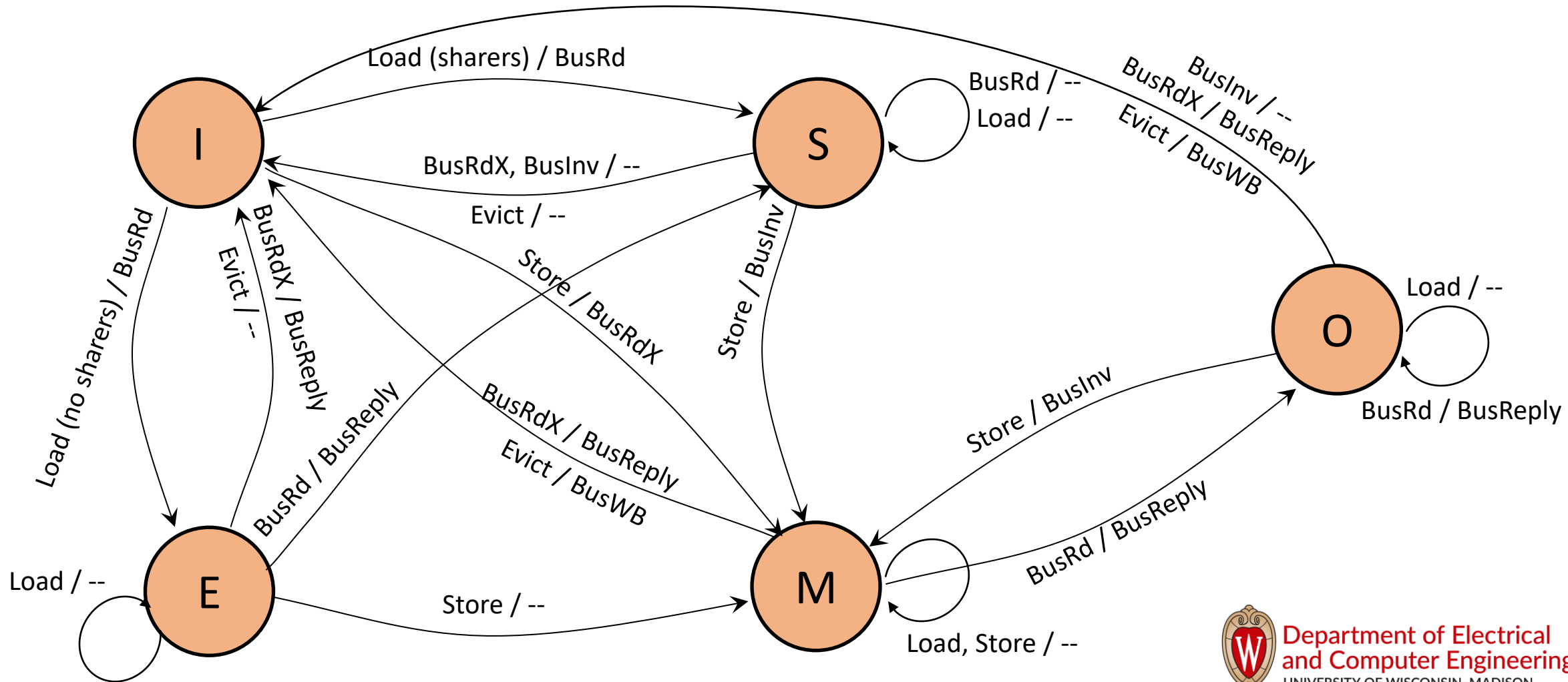
In MESIF below, why is there no transition from S to F?

**F only for most recent requester
(who was previously invalid)**



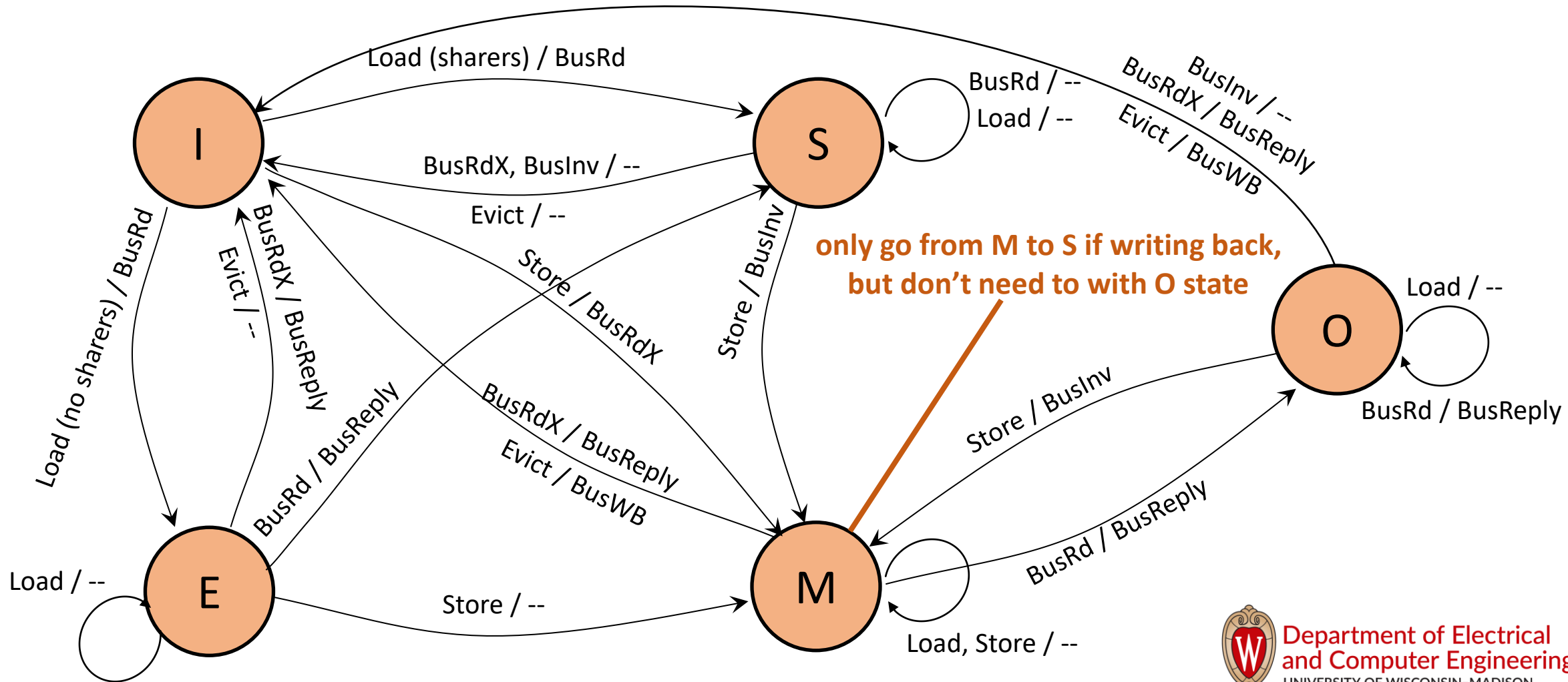
Snooping-Based Coherence

In MOESI below, why is there no transition from M to S?



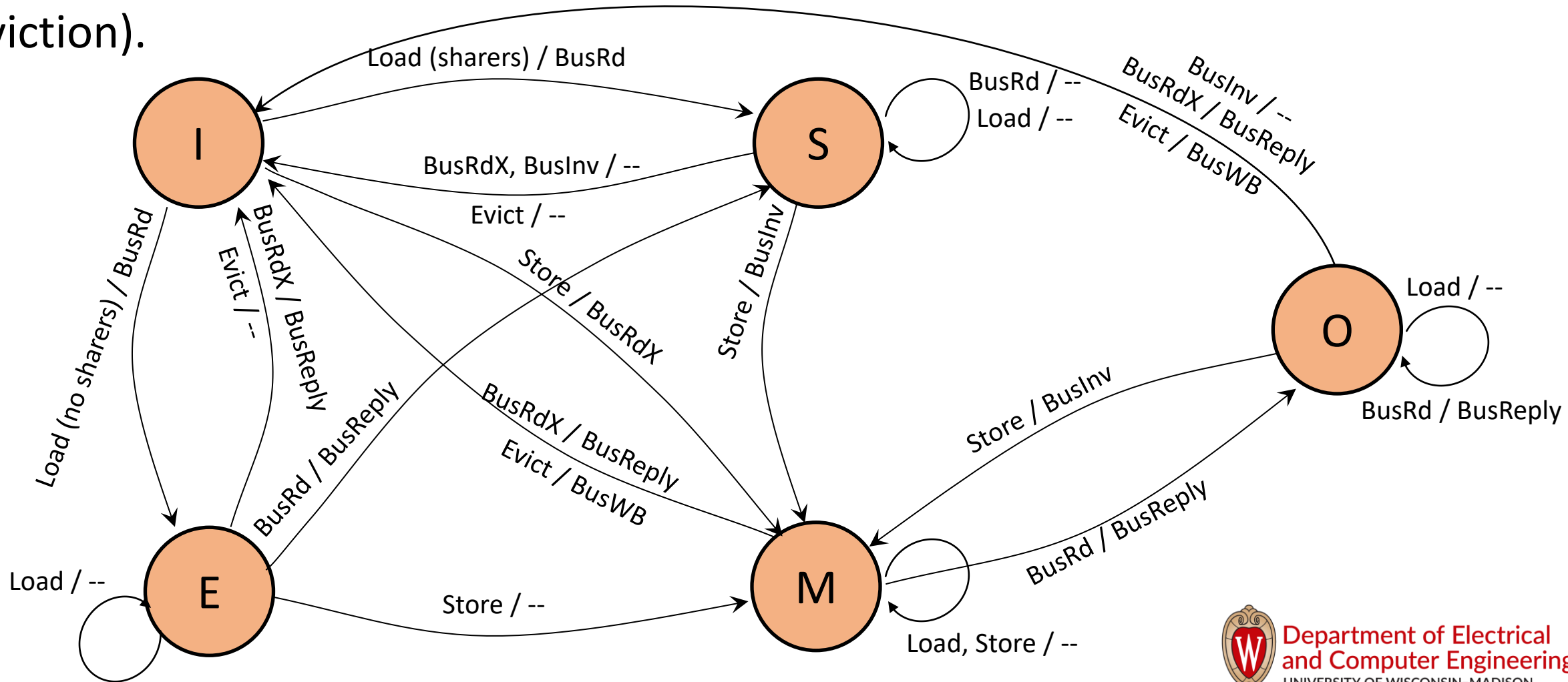
Snooping-Based Coherence

In MOESI below, why is there no transition from M to S?



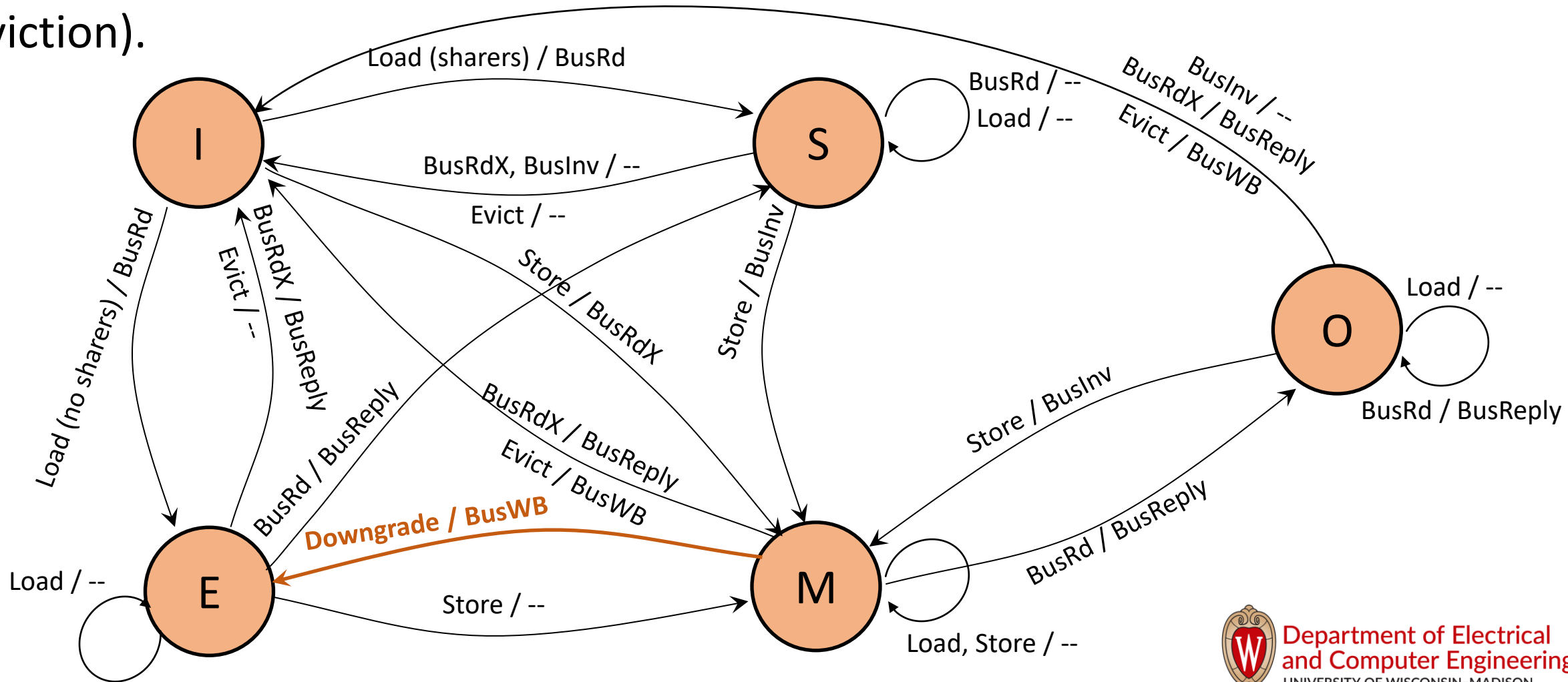
Snooping-Based Coherence

Modify MOESI to support a new *Downgrade* event (i.e., write-back without eviction).



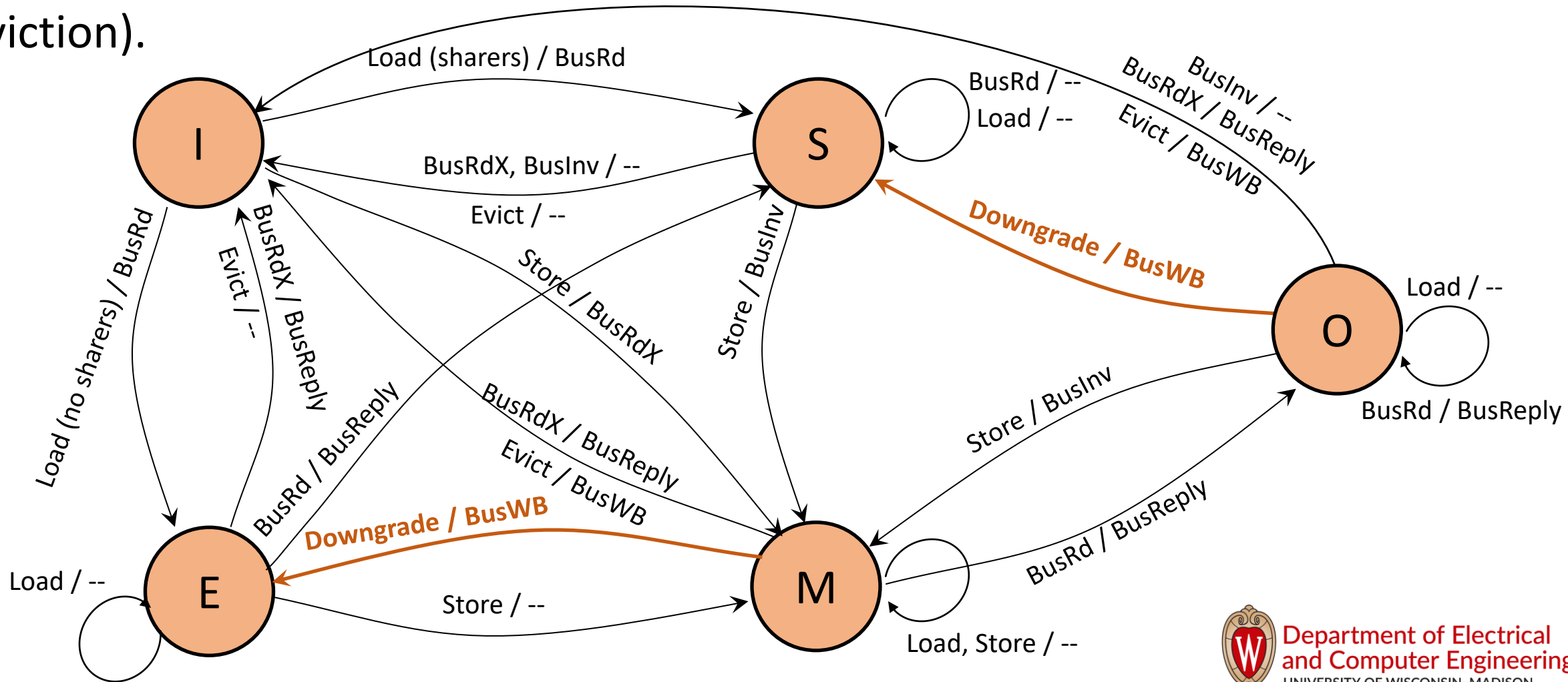
Snooping-Based Coherence

Modify MOESI to support a new *Downgrade* event (i.e., write-back without eviction).



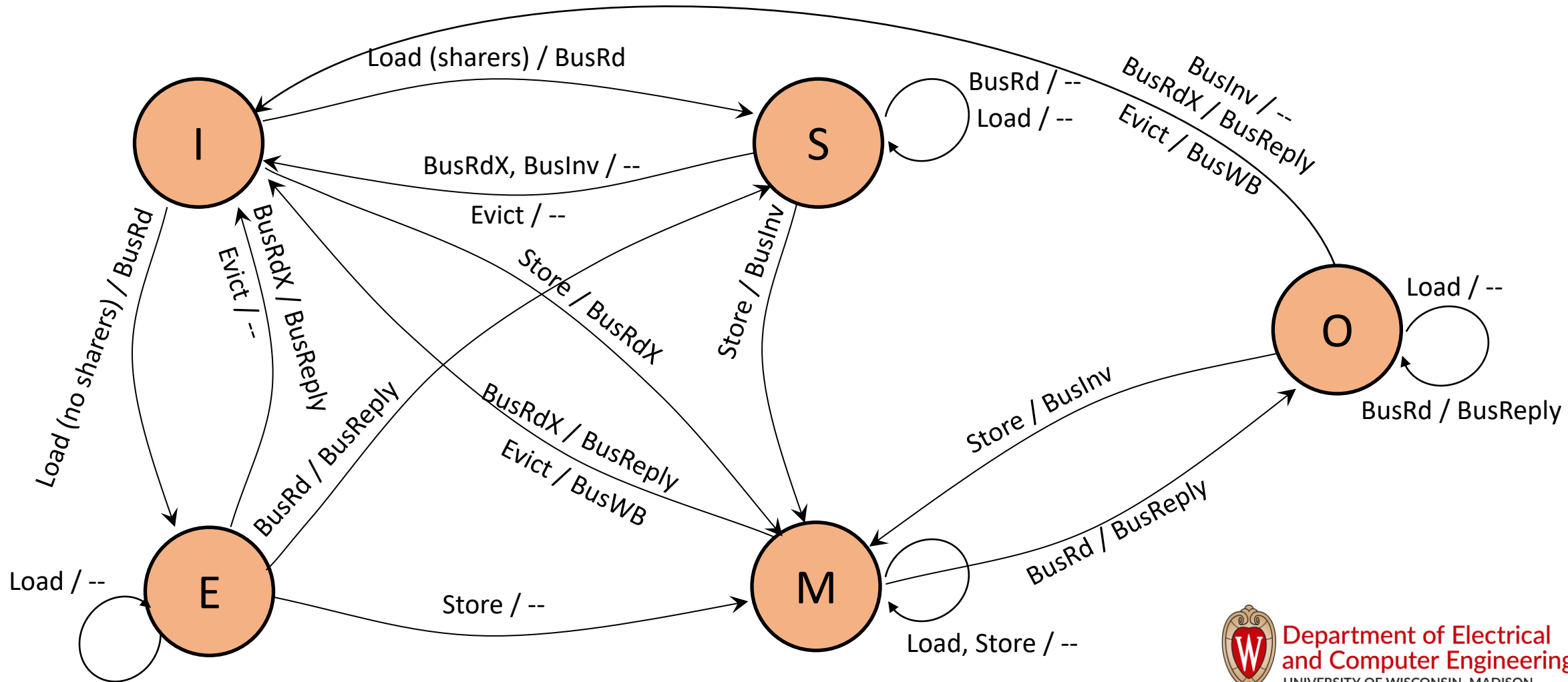
Snooping-Based Coherence

Modify MOESI to support a new *Downgrade* event (i.e., write-back without eviction).



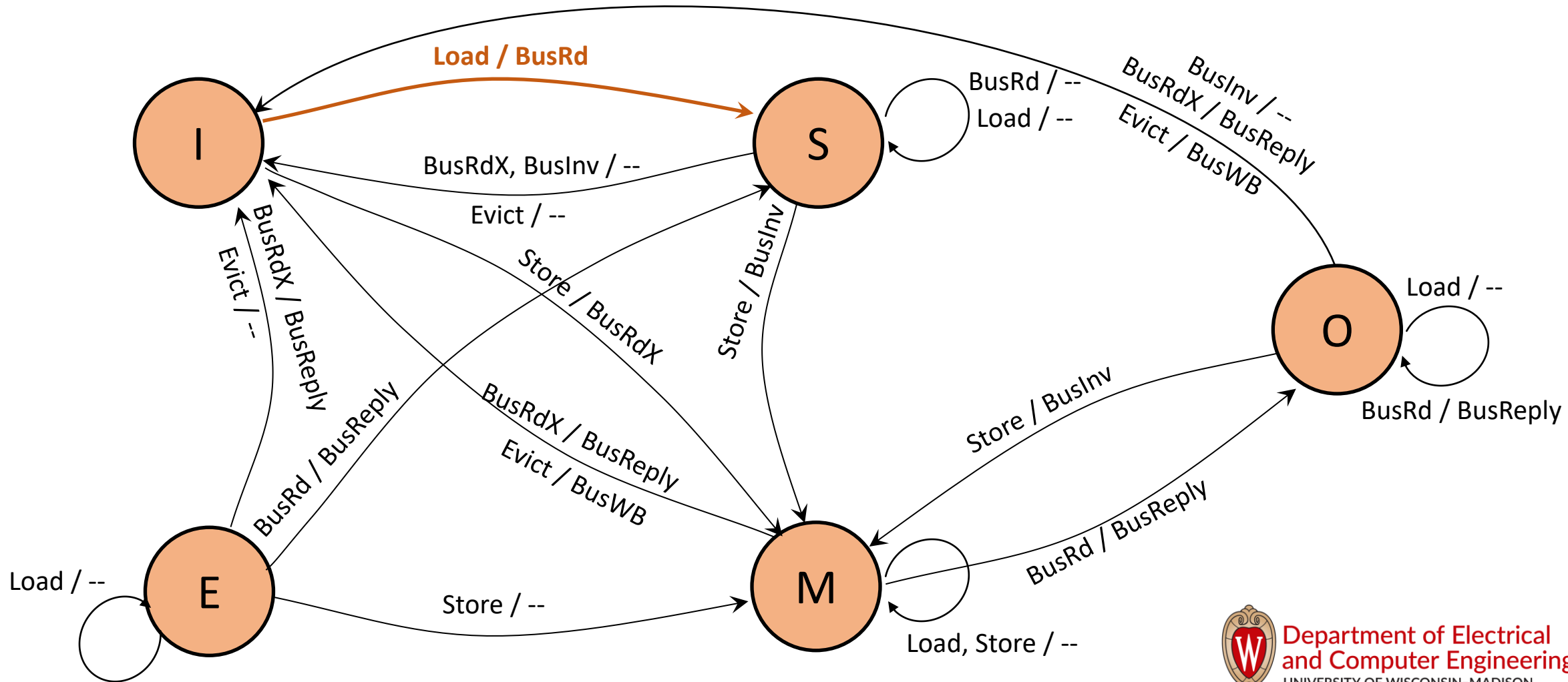
Snooping-Based Coherence

Convert the following MOESI protocol to MOSI.



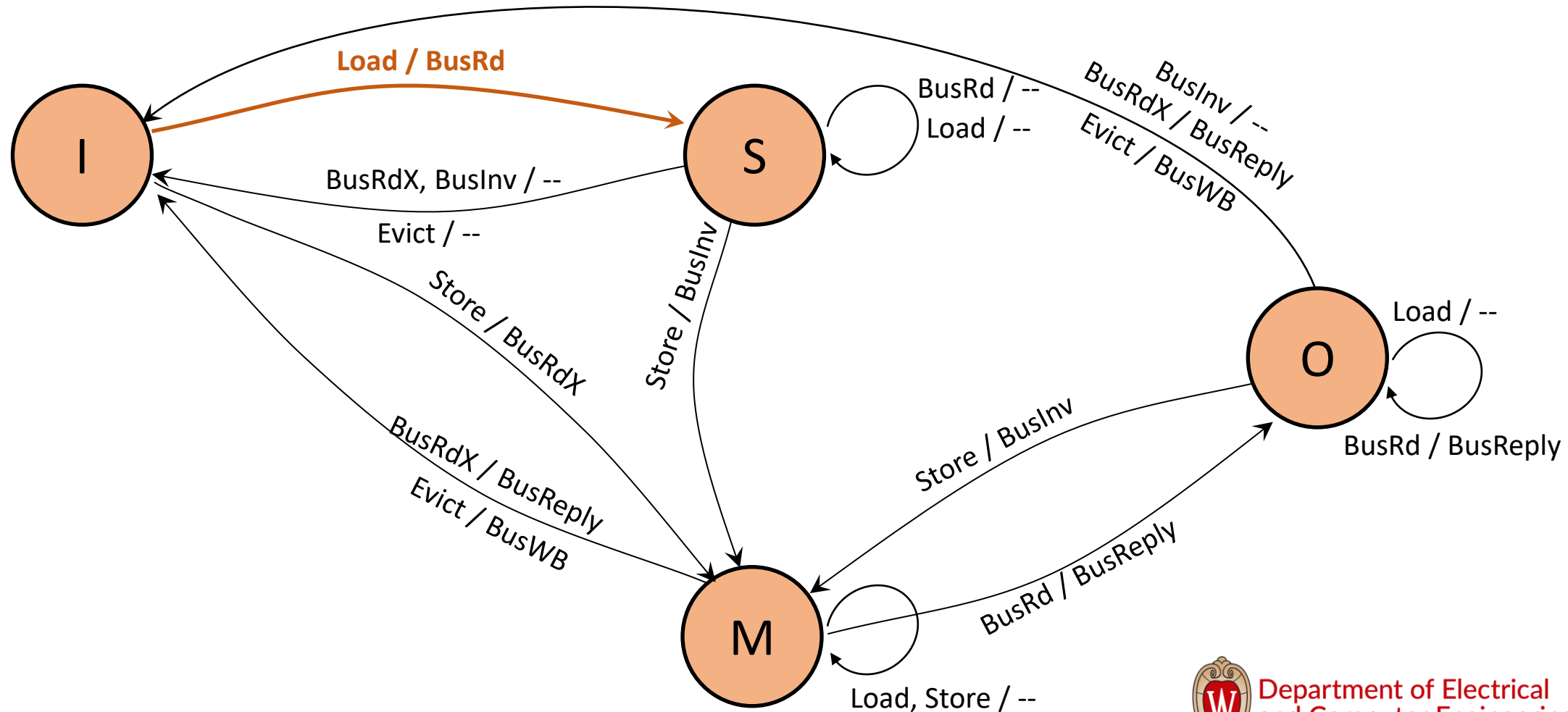
Snooping-Based Coherence

Convert the following MOESI protocol to MOSI.



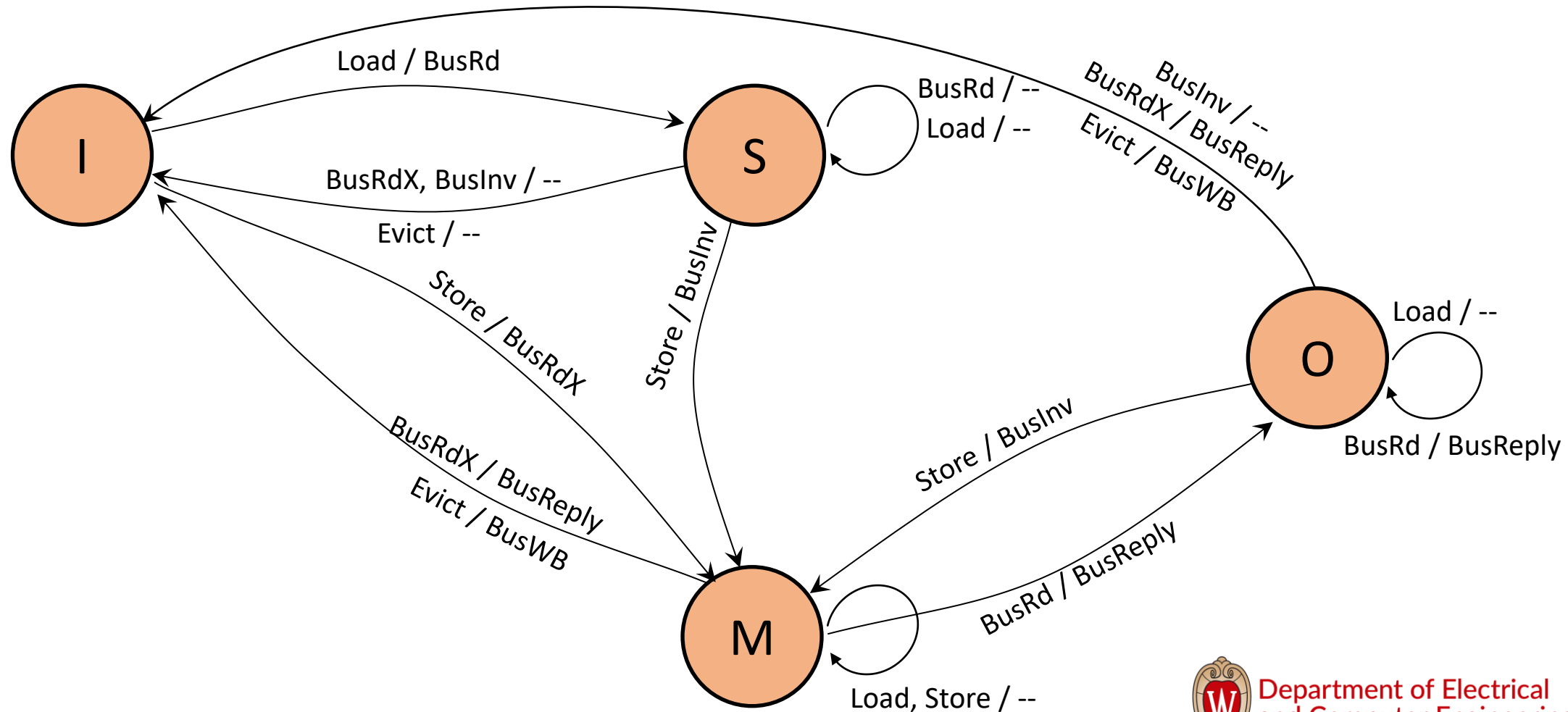
Snooping-Based Coherence

Convert the following MOESI protocol to MOSI.



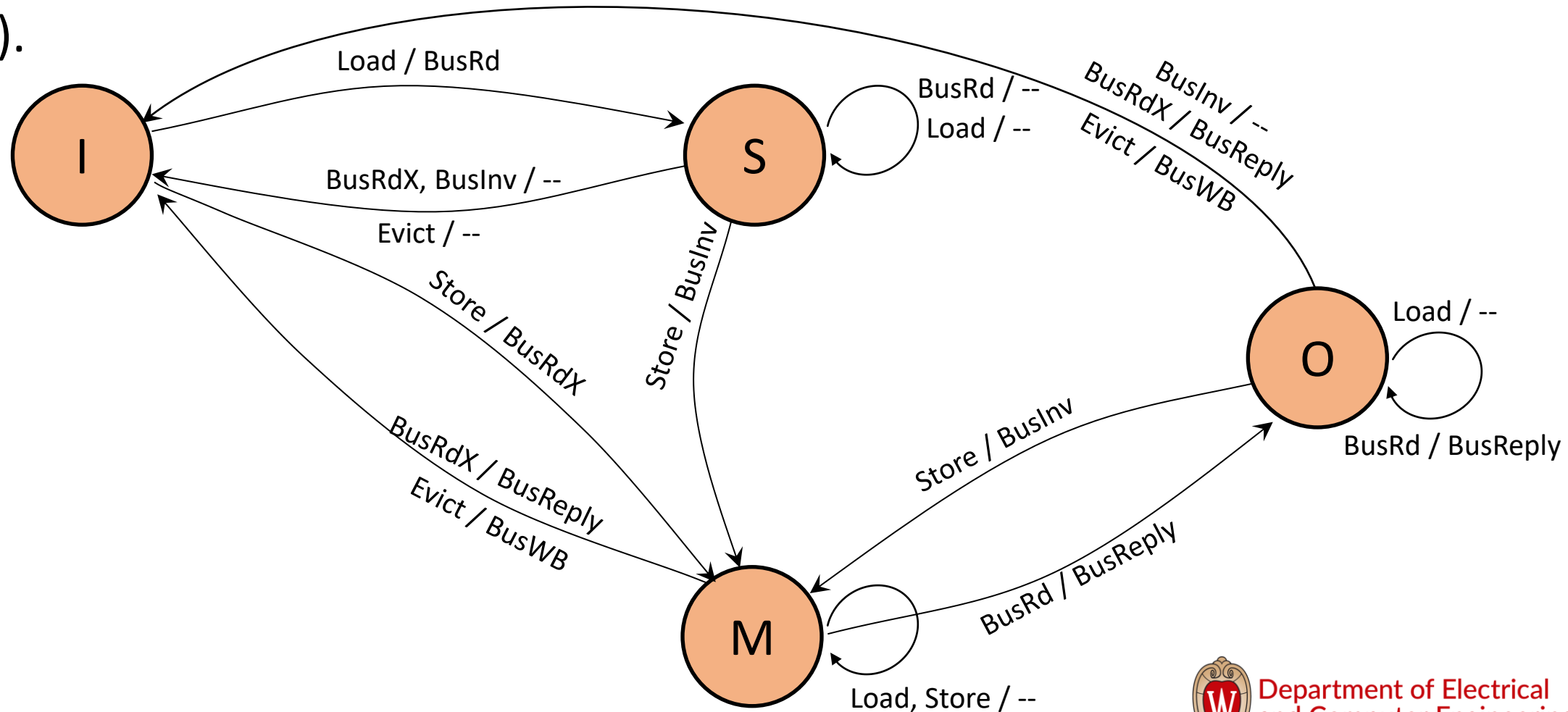
Snooping-Based Coherence

Convert the following MOESI protocol to MOSI.



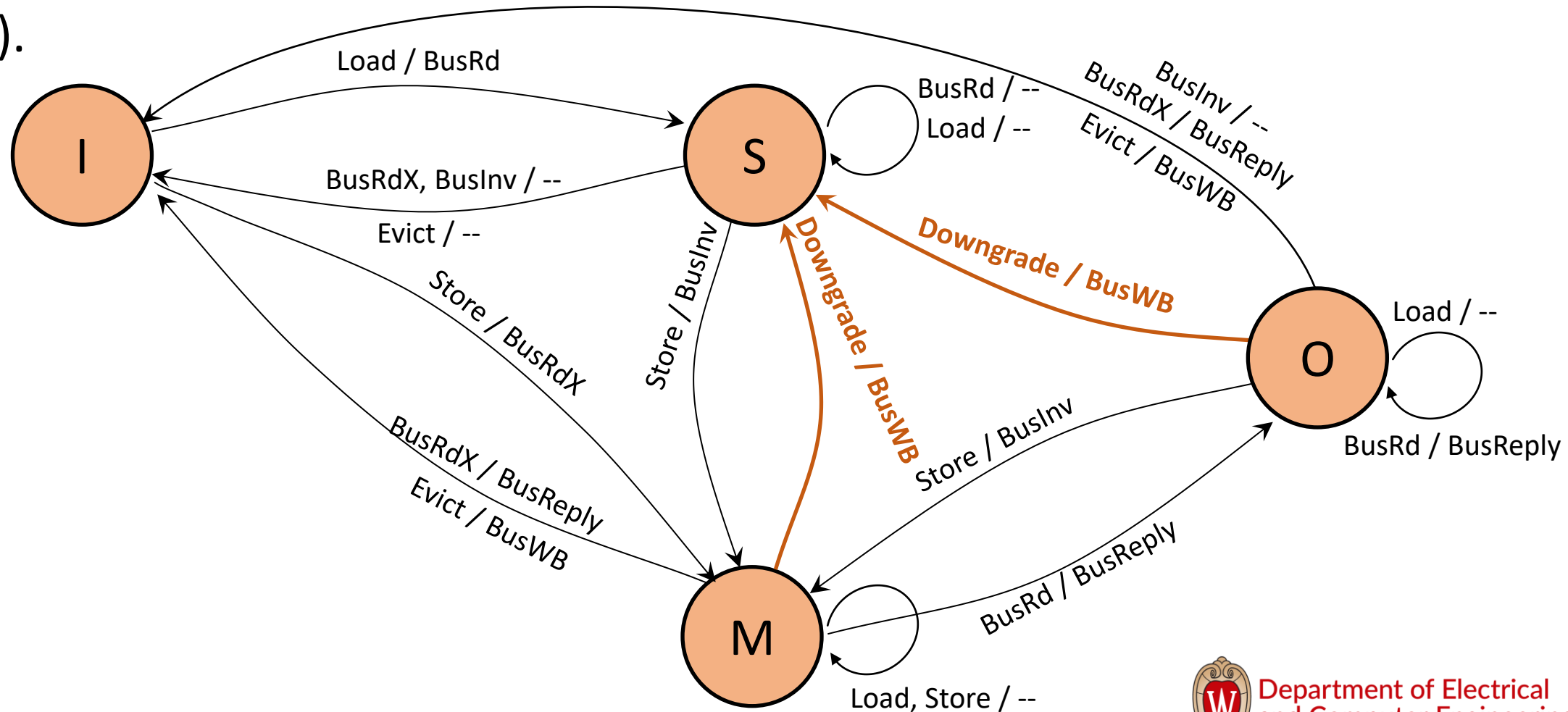
Snooping-Based Coherence

Modify MOSI to support a new *Downgrade* event (i.e., write-back without eviction).



Snooping-Based Coherence

Modify MOSI to support a new *Downgrade* event (i.e., write-back without eviction).



Directory-Based Coherence

In the MSI protocol discussed in class, how is it possible for the directory controller to receive a PUTS-Last request in the M state?

Directory Controller

Current State	Event						
	GetS	GetM	PutS (not last)	PutS (last)	PutM (owner)	PutM (not owner)	Data
I	Data to req → S	Data to req → M	Put-Ack to req → I	Put-Ack to req → I		Put-Ack to req → I	
S	Data to req → S	Data to req Inv to sharers → M	Put-Ack to req → S	Put-Ack to req → I		Put-Ack to req → S	
M	Fwd-GetS to owner → S ^D	Fwd-GetM to owner → M	Put-Ack to req → M	Put-Ack to req → M	Data to mem Put-Ack to req → I	Put-Ack to req → M	
S ^D	stall	stall	Put-Ack to req → S ^D	Put-Ack to req → S ^D		Put-Ack to req → S ^D	Data to mem → S



Directory-Based Coherence

In the MSI protocol discussed in class, how is it possible for the directory controller to receive a PUTS-Last request in the M state?

Directory Controller

Current State	Event						
	GetS	GetM	PutS (not last)	PutS (last)	PutM (owner)	PutM (not owner)	Data
I	Data to req → S	Data to req → M	Put-Ack to req → I	Put-Ack to req → I		Put-Ack to req → I	
S	Data to req → S	Data to req Inv to sharers → M	Put-Ack to req → S	Put-Ack to req → I		Put-Ack to req → S	
M	Fwd-GetS to owner → S ^D	Fwd-GetM to owner → M	Put-Ack to req → M	Put-Ack to req → M	Data to mem Put-Ack to req → I	Put-Ack to req → M	
S ^D	stall	stall	Put-Ack to req → S ^D	Put-Ack to req → S ^D		Put-Ack to req → S ^D	Data to mem → S

directory transitions to M immediately; meanwhile sharers may be evicting

Directory-Based Coherence

Why is it impossible for a cache controller to get a Fwd-GetS in SI^A state?

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS^D	GetM to dir → IM^{AD}										
IS^D	stall	stall	stall			stall		→ S		→ S		
IM^{AD}	stall	stall	stall	stall	stall			→ M	→ IM^A	→ M	ack-- → IM^{AD}	
IM^A	stall	stall	stall	stall	stall						ack-- → IM^A	→ M
S	hit → S	GetM to dir → SM^{AD}	PutS to dir → SI^A			Inv-Ack to req → I						
SM^{AD}	hit → SM^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM^{AD}		→ M	→ SM^A		ack-- → SM^{AD}	
SM^A	hit → SM^A	stall	stall	stall	stall						ack-- → SM^A	→ M
M	hit → M	hit → M	PutM to dir → MI^A	Data to req+dir → S	Data to req → I							
MI^A	stall	stall	stall	Data to req+dir → SI^A	Data to req → II^A		→ I					
SI^A	stall	stall	stall			Inv-Ack to req → II^A	→ I					
II^A	stall	stall	stall				→ I					

Directory-Based Coherence

Why is it impossible for a cache controller to get a Fwd-GetS in SI^A state?

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS^D	GetM to dir → IM^{AD}										
IS^D	stall	stall	stall			stall		→ S		→ S		
IM^{AD}	stall	stall	stall	stall	stall			→ M	→ IM^A	→ M	ack-- → IM^{AD}	
IM^A	stall	stall	stall	stall	stall						ack-- → IM^A	→ M
S	hit → S	GetM to dir → SM^{AD}	PutS to dir → SI^A			Inv-Ack to req → I						
SM^{AD}	hit → SM^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM^{AD}		→ M	→ SM^A		ack-- → SM^{AD}	
SM^A	hit → SM^A	stall	stall	stall	stall						ack-- → SM^A	→ M
M	hit → M	hit → M	PutM to dir → MI^A	Data to req+dir → S	Data to req → I							
MI^A	stall	stall	stall	Data to req+dir → SI^A	Data to req → II^A		→ I					
SI^A	stall	stall	stall			Inv-Ack to req → II^A	→ I					
II^A	stall	stall	stall				→ I					

if from here,
no Owner since Shared

Directory-Based Coherence

Why is it impossible for a cache controller to get a Fwd-GetS in SI^A state?

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS^D	GetM to dir → IM^{AD}										
IS^D	stall	stall	stall			stall		→ S		→ S		
IM^{AD}	stall	stall	stall	stall	stall			→ M	→ IM^A	→ M	ack-- → IM^{AD}	
IM^A	stall	stall	stall	stall	stall						ack-- → IM^A	→ M
S	hit → S	GetM to dir → SM^{AD}	PutS to dir → SI^A			Inv-Ack to req → I						
SM^{AD}	hit → SM^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM^{AD}		→ M	→ SM^A		ack-- → SM^{AD}	
SM^A	hit → SM^A	stall	stall	stall	stall						ack-- → SM^A	→ M
M	hit → M	hit → M	PutM to dir → MI^A	Data to req+dir → S	Data to req → I							
MI^A	stall	stall	stall	Data to req+dir → SI^A	Data to req → II^A		→ I					
SI^A	stall	stall	stall			Inv-Ack to req → II^A	→ I					
II^A	stall	stall	stall				→ I					

if from here,
Dir just cleared Owner

Directory-Based Coherence

Why must we stall instead of hit upon a Load in MI^A state?

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS^D	GetM to dir → IM^{AD}										
IS^D	stall	stall	stall			stall		→ S		→ S		
IM^{AD}	stall	stall	stall	stall	stall			→ M	→ IM^A	→ M	ack-- → IM^{AD}	
IM^A	stall	stall	stall	stall	stall						ack-- → IM^A	→ M
S	hit → S	GetM to dir → SM^{AD}	PutS to dir → SI^A			Inv-Ack to req → I						
SM^{AD}	hit → SM^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM^{AD}		→ M	→ SM^A		ack-- → SM^{AD}	
SM^A	hit → SM^A	stall	stall	stall	stall						ack-- → SM^A	→ M
M	hit → M	hit → M	PutM to dir → MI^A	Data to req+dir → S	Data to req → I							
MI^A	stall	stall	stall	Data to req+dir → SI^A	Data to req → II^A		→ I					
SI^A	stall	stall	stall			Inv-Ack to req → II^A	→ I					
II^A	stall	stall	stall				→ I					

Directory-Based Coherence

Why must we stall instead of hit upon a Load in MI^A state?

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS^D	GetM to dir → IM^{AD}										
IS^D	stall	stall	stall			stall		→ S		→ S		
IM^{AD}	stall	stall	stall	stall	stall			→ M	→ IM^A	→ M	ack-- → IM^{AD}	
IM^A	stall	stall	stall	stall	stall						ack-- → IM^A	→ M
S	hit → S	GetM to dir → SM^{AD}	PutS to dir → SI^A			Inv-Ack to req → I						
SM^{AD}	hit → SM^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM^{AD}		→ M	→ SM^A		ack-- → SM^{AD}	
SM^A	hit → SM^A	stall	stall	stall	stall						ack-- → SM^A	→ M
M	hit → M	hit → M	PutM to dir → MI^A	Data to req+dir → S	Data to req → I							
MI^A	stall	stall	stall	Data to req+dir → SI^A	Data to req → II^A		→ I					
SI^A	stall	stall	stall			Inv-Ack to req → II^A	→ I					
II^A	stall	stall	stall				→ I					

PutM may have arrived at Dir, so maybe new writer